I am a beginner at reverse engineering. Im a second year college student, so if this is a bit illegible i apologize.

Opening the file in ghidra, after analysis we find theres no main function. But at the entry point **__libc_start_main** is called so we can easily find the main function from here.

```
void processEntry entry(undefined8 param_1,undefined8 param_2)

{
  undefined1 auStack_8 [8];

  __libc_start_main(FUN_00101445,param_2,&stack0x00000008,0,0,param_1,auStack_8);
  do {
                    /* WARNING: Do nothing block with infinite loop */
  } while( true );
}
```

Double clicking on **FUN_00101445** will take us to the main function

```
undefined8 main(void)

{
  int iVar1;
  int iVar2;
  undefined8 uVar3;
  size_t sVar4;
  time_t tVar5;
  long in_FS_OFFSET;
  undefined4 local_64;
  undefined1 local_60;
  code *local_50;
  char local_3a;
  char local_38 [40];
  long local_10;

  local_10 = *(long *)(in_FS_OFFSET + 0x28);
  iVar1 = FUN_00101258();
  if (iVar1 == 0) {
```

The first line of note, is the function call, and the if statement that depends on the outcome of the function call.

decompiling this function shows us this:

```
bool FUN_00101258(void)

{
  int iVar1;
  FILE *__stream;
  char *pcVar2;
  long in_FS_OFFSET;
  bool bVar3;
  char local_118 [10];
  char acStack_10e [254];
  long local_10;

  local_10 = *(long *)(in_FS_OFFSET + 0x28);
  __stream = fopen("/proc/self/status","r");
  if (__stream == (FILE *)0x0) {
    bVar3 = false;
  }
  else {
    do {
      pcVar2 = fgets(local_118,0x100,__stream);
      if (pcVar2 == (char *)0x0) {
        fclose(__stream);
        bVar3 = false;
        goto LAB_00101333;
      }
      iVar1 = strncmp(local_118,"TracerPid:",10);
    } while (iVar1 != 0);
    iVar1 = atoi(acStack_10e);
    fclose(__stream);
    bVar3 = iVar1 != 0;
  }
LAB_00101333:
  if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
```

As complicated as this looks, we can see theres only a single return statement, meaning all we need to do is follow this variable and see where its assigned.
The most important line is **15,** opening **/proc/self/status** this file contains info about the current process. Using line 21, and 27, shows that its looking for **"TracerPID:"** Meaning its looking to see if the process is being ran by a **debugger**. Also confirmed by the main function, if it detects the process

```
44  else {
45    puts("Noooooooo. La Policiaaa.");
```

This means that **FUN_00101258()** can be renamed to **isDebuggerPresent()**

Continuing on with the main function, there are variable assignments, and another function call. Since this crackme has control flow obfuscation, were not going to look at these too closely, unless they come up later.

```
iVar1 = rand();
local_64 = 0x746f6f72;
local_60 = 0;
local_50 = FUN_00101349;
```

However the function call is of note, since local_50 is used later in the program so were going to follow that.

```
1
2  bool FUN_00101349(char *param_1)
3
4  {
5    int iVar1;
6    long in_FS_OFFSET;
7    char local_28 [24];
8    long local_10;
9
10   local_10 = *(long *)(in_FS_OFFSET + 0x28);
11   strcpy(local_28,&DAT_00102023);
12   FUN_00101209(local_28);
13   iVar1 = strcmp(param_1,local_28);
14   if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
15                 /* WARNING: Subroutine does not return */
16     __stack_chk_fail();
17   }
18   return iVar1 == 0;
19 }
```

In **strcopy**, some info goes into **local_28**, using ghidra to follow that address, we see its a series of hex characters

```
                    DAT_00102023

00102023 dd          ??         DDh
00102024 da          ??         DAh
00102025 f6          ??         F6h
00102026 d9          ??         D9h
00102027 c4          ??         C4h
00102028 c6          ??         C6h
00102029 f6          ??         F6h
0010202a ce          ??         CEh
0010202b c7          ??         C7h
0010202c ce          ??         CEh
0010202d f6          ??         F6h
0010202e c0          ??         C0h
0010202f ca          ??         CAh
00102030 c5          ??         C5h
00102031 00          ??         00h
```

Consider the name xorcist, this is likely going to be used to get the flag one way or another. However we dont know yet, so we will come back to this.

Continuing on we see **local_28** gets passed to a function, which when decompiling we see this:

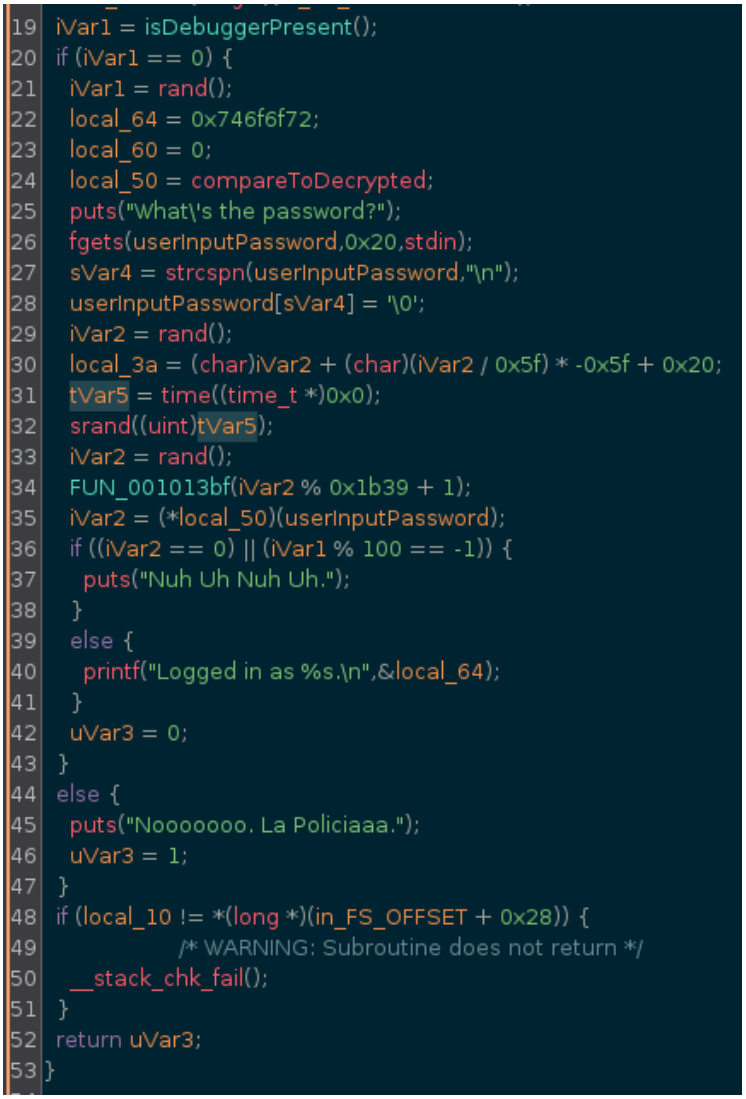
```
1
2 void FUN_00101209(long input)
3
4 {
5  undefined4 iterator;
6
7  for (iterator = 0; *(char *)(input + iterator) != '\0'; iterator = iterator + 1) {
8    *(byte *)(input + iterator) = *(byte *)(input + iterator) ^ 0xa9;
9  }
10  return;
11 }
12
```

I have renamed the variables for easier reading. This is a fairly classic function in CTFS, being either a decrypt or encrypt function. Considering that the parameter for this function is a string of hex characters from the last function, this is likely a **decode** function. So I will rename it for clarity.

After looking through the decrypt function, we can continue on with function that called it

```
12  decrypt(local_28);
13  iVar1 = strcmp(param_1,local_28);
14  if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
15                  /* WARNING: Subroutine does not return */
16    __stack_chk_fail();
17  }
18  return iVar1 == 0;
19 }
20
```

After the call to the decrypt function, it returns a comparison meaning this function checks the input, to the assumedly decrypted series of hex characters, likely the flag were going to have to find. I will rename this function to **compareToDecrypt** for clarity.



```
19  iVar1 = isDebuggerPresent();
20  if (iVar1 == 0) {
21    iVar1 = rand();
22    local_64 = 0x746f6f72;
23    local_60 = 0;
24    local_50 = compareToDecrypted;
25    puts("What\'s the password?");
26    fgets(userInputPassword,0x20,stdin);
27    sVar4 = strcspn(userInputPassword,"\n");
28    userInputPassword[sVar4] = '\0';
29    iVar2 = rand();
30    local_3a = (char)iVar2 + (char)(iVar2 / 0x5f) * -0x5f + 0x20;
31    tVar5 = time((time_t *)0x0);
32    srand((uint)tVar5);
33    iVar2 = rand();
34    FUN_001013bf(iVar2 % 0x1b39 + 1);
35    iVar2 = (*local_50)(userInputPassword);
36    if ((iVar2 == 0) || (iVar1 % 100 == -1)) {
37      puts("Nuh Uh Nuh Uh.");
38    }
39    else {
40      printf("Logged in as %s.\n",&local_64);
41    }
42    uVar3 = 0;
43  }
44  else {
45    puts("Nooooooo. La Policiaaa.");
46    uVar3 = 1;
47  }
48  if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
49                  /* WARNING: Subroutine does not return */
50    __stack_chk_fail();
51  }
52  return uVar3;
53}
```

Back to the main function, at line 26 the program gets input from the command line, which I renamed to **userInputPassword**, and as usual with ghidra, changes the newline terminator to a null value

Then theres a bunch of bullshit, **iVar2, local_3a, and tVar5** are all just obfuscation. This is most apparent as they're assigned and not used. **iVar2** is given a random value, and then assigned again after.

Line 34 isn't even worth mentioning, as it has no return, and again **iVar2** gets reassigned right after, so its another distraction function.

Line 35 might be a little confusing, at first glance, it looks like **userInputPassword** is being cast to the type (**local_50)** but if we look further up the code, at line 13, and 24, we can see **local_50** is a pointer to a function, meaning that its actually a function call to **compareToDecrypt**, so **iVar2** is assigned to the result of **compareToDecrypt(userInputPassword)**, line 36, 37 and 40 being indicative of a check, and a fail or pass. The second condition is a decompilation error by ghidra. Reading the assembly shows it fails if **iVar1** is equal to 0.

The key is finding what **userInputPassword** has to be equal to. Back to the **compareToDecrypted** function.

```
bool compareToDecrypted(char *param_1)

{
 int iVar1;
 long in_FS_OFFSET;
 char local_28 [24];
 long local_10;

 local_10 = *(long *)(in_FS_OFFSET + 0x28);
 strcpy(local_28,&DAT_00102023);
 decrypt(local_28);
 iVar1 = strcmp(param_1,local_28);
```

Seeing this, and following what **DAT_00102023** is in memory, a series of hex characters are passed to the decrypt function. As we saw earlier, its equal to **"\xdd\xda\xf6\xd9\xc4\xc6\xf6\xce\xc7\xce\xf6\xc0\xca\xc5"** which get XOR'd with **0xa9** or 169d
Writing a simple python program we can decrypt these hex characters, giving us a string of numbers. Since the input is a password, its clear these are character values.

```
characters = [0xdd, 0xda, 0xf6, 0xd9, 0xc4, 0xc6,
0xf6, 0xce, 0xc7, 0xce, 0xf6, 0xc0, 0xca, 0xc5]

for int in characters:
    print(chr(int ^ 169))
```

After running the program we get the password "**ts_pmo_gng_icl**"

```
$./xorcist
What's the password?
ts_pmo_gng_icl
Logged in as root.
```

Entering the password we log in as root and solve the CTF!